MYSQL PROJECT

TOPIC: eCHALLAN TRAFFIC ENFORCEMENT & VIOLATION ANALYSIS USING MYSQL

SOURCE:KAGGLE

INTRODUCTION

This project performs a structured analysis of an eChallan traffic violation dataset using MySQL. The dataset contains daily information related to total challans, disposed challans, pending challans, challan amounts, and court cases.

The objective of this analysis is to extract meaningful insights related to challan activity, disposal efficiency, pending financial burden, and court-case activity using SQL techniques such as filtering, aggregation, sorting, grouping, arithmetic operations, and subqueries.

TOOL: MySQL Workbench

Submitted By – Gouri PS

Queries :

1.Find the dates where pending challans were greater than 100 and pending court cases were greater than 2.

SELECT date, pendingChallan, pendingCourt FROM violations WHERE pendingChallan > 100 AND pendingCourt > 2;



Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

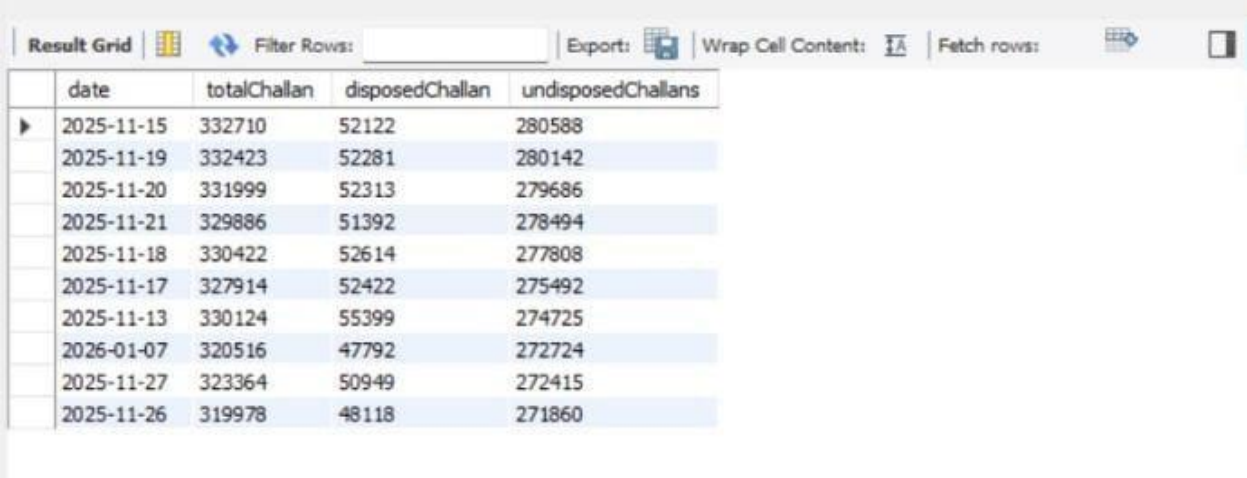
date	pendingChallan	pendingCourt
2015-01-09	107	7
2015-01-19	102	7
2015-01-20	104	7
2015-01-28	111	4
2015-02-10	136	5
2015-02-14	106	3
2015-02-20	106	3
2015-03-03	117	6
2015-03-04	101	4
2015-03-09	101	4
2015-03-10	101	5
2015-03-18	119	3

Insight: Several dates recorded more than 100 pending challans along with more than two pending court cases. This helps identify periods with higher unresolved enforcement and court activity.

2.Display the 10 dates having the highest difference between total challans and disposed challans.

SELECT date, totalChallan, disposedChallan,

(totalChallan - disposedChallan) AS undisposedChallans FROM violations ORDER BY undisposedChallans DESC LIMIT 10;



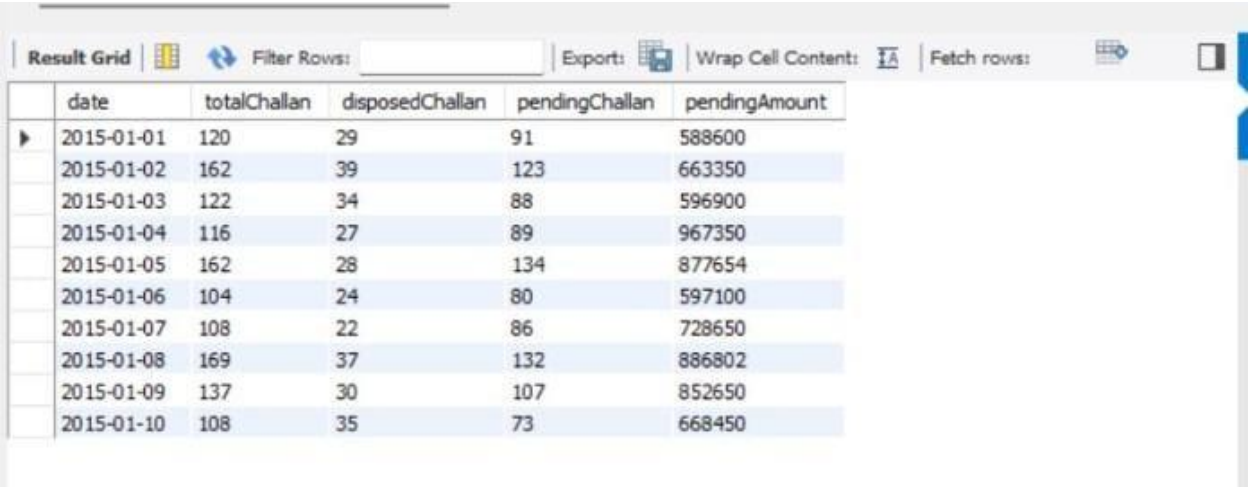
Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

date	totalChallan	disposedChallan	undisposedChallans
2025-11-15	332710	52122	280588
2025-11-19	332423	52281	280142
2025-11-20	331999	52313	279686
2025-11-21	329886	51392	278494
2025-11-18	330422	52614	277808
2025-11-17	327914	52422	275492
2025-11-13	330124	55399	274725
2026-01-07	320516	47792	272724
2025-11-27	323364	50949	272415
2025-11-26	319978	48118	271860

Insight: The difference between total and disposed challans varied across dates, with some dates showing a much larger number of undisposed challans. This helps identify periods with higher pending workload

3.Retreive the first 10 eChallan records showing Date, Total Challans, Disposed Challans, Pending Challans, and Pending Amount.

SELECT date, totalChallan, disposedChallan, pendingChallan, pendingAmount FROM violations LIMIT 10;



Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

date	totalChallan	disposedChallan	pendingChallan	pendingAmount
2015-01-01	120	29	91	588600
2015-01-02	162	39	123	663350
2015-01-03	122	34	88	596900
2015-01-04	116	27	89	967350
2015-01-05	162	28	134	877654
2015-01-06	104	24	80	597100
2015-01-07	108	22	86	728650
2015-01-08	169	37	132	886802
2015-01-09	137	30	107	852650
2015-01-10	108	35	73	668450

Insight: The first 10 records provide an overview of daily challan activity, including total, disposed, and pending challans along with the pending financial amount.

4. Calculate the total, average, highest, and lowest pending amount recorded in the dataset.

SELECT SUM(pendingAmount) AS total_pending_amount, AVG(pendingAmount) AS average_pending_amount, MAX(pendingAmount) AS highest_pending_amount, MIN(pendingAmount) AS lowest_pending_amount FROM violations;

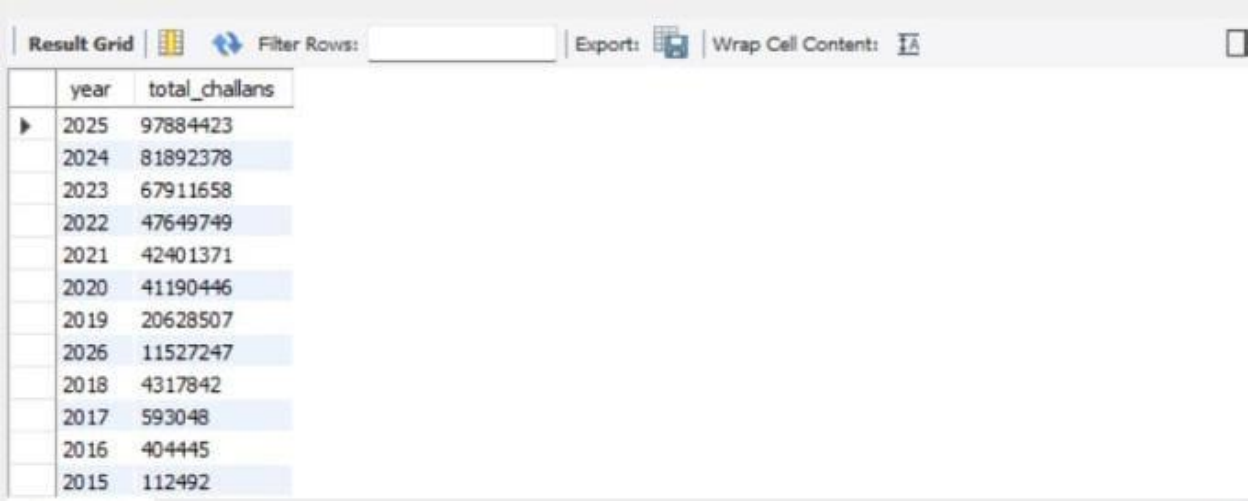


total_pending_amount	average_pending_amount	highest_pending_amount	lowest_pending_amount
406026013895	99981781.3088	565996093	20400

Insight: Pending amounts varied considerably across the dataset, with differences between the total, average, highest, and lowest amounts. This helps assess the overall financial burden of unresolved challans.

5. Calculate the total challans for each year and display the years in descending order of total challans.

SELECT YEAR(date) AS year,SUM(totalChallan) AS total_challans FROM violations GROUP BY YEAR(date) ORDER BY total_challans DESC;



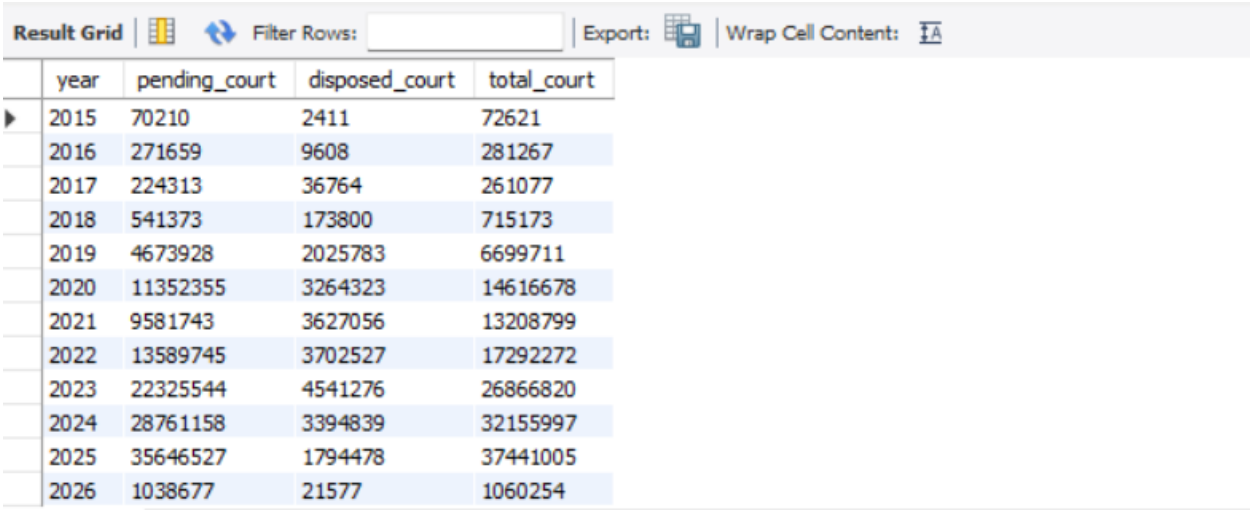
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

year	total_challans
2025	97884423
2024	81892378
2023	67911658
2022	47649749
2021	42401371
2020	41190446
2019	20628507
2026	11527247
2018	4317842
2017	593048
2016	404445
2015	112492

Insight: The total number of challans varied across years, with some years recording higher challan volumes than others. This helps identify years with greater traffic enforcement activity.

6.Calculate the total pending, disposed, and overall court cases for each year.

SELECT YEAR(date) AS year, SUM(pendingCourt) AS pending_court, SUM(disposedCourt) AS disposed_court,SUM(totalCourt) AS total_court FROM violations GROUP BY YEAR(date) ORDER BY year;



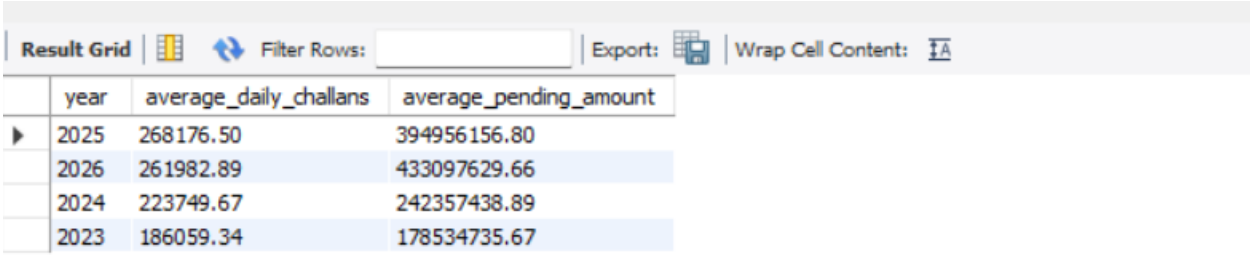
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

year	pending_court	disposed_court	total_court
2015	70210	2411	72621
2016	271659	9608	281267
2017	224313	36764	261077
2018	541373	173800	715173
2019	4673928	2025783	6699711
2020	11352355	3264323	14616678
2021	9581743	3627056	13208799
2022	13589745	3702527	17292272
2023	22325544	4541276	26866820
2024	28761158	3394839	32155997
2025	35646527	1794478	37441005
2026	1038677	21577	1060254

Insight: Pending, disposed, and total court cases varied across years. This helps understand changes in court-related activity and unresolved cases over time.

7. Identify the years with the highest average daily challan volume among years whose average pending amount was higher than the overall average.

```
SELECT YEAR(date) AS year, ROUND(AVG(totalChallan), 2) AS
average_daily_challans, ROUND(AVG(pendingAmount), 2) AS
average_pending_amount FROM violations GROUP BY YEAR(date)
HAVING AVG(pendingAmount) > ( SELECT AVG(pendingAmount)
FROM violations ) ORDER BY average_daily_challans DESC;
```

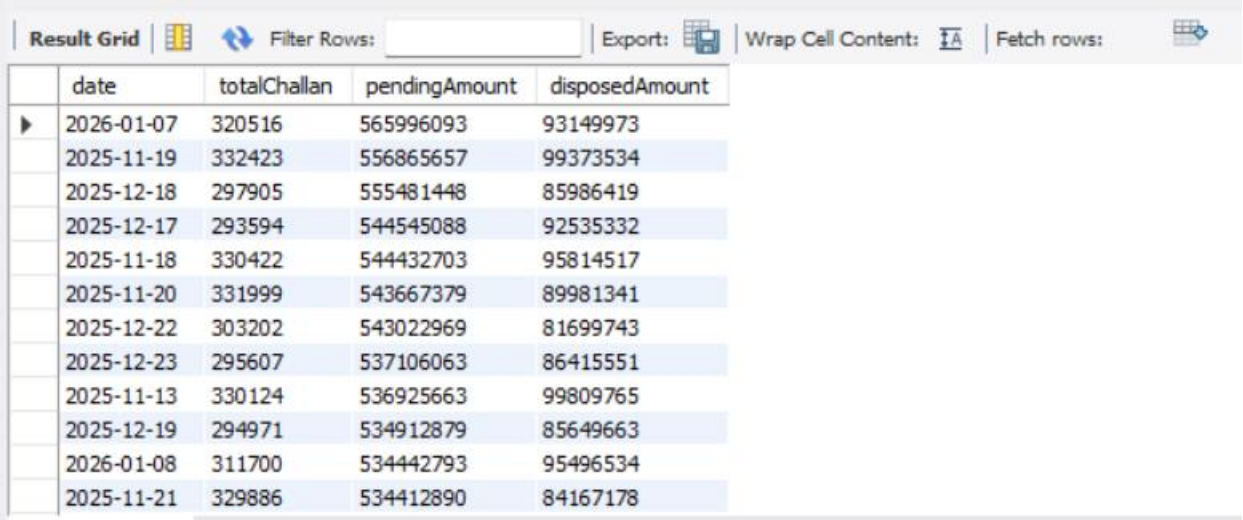

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

year	average_daily_challans	average_pending_amount
2025	268176.50	394956156.80
2026	261982.89	433097629.66
2024	223749.67	242357438.89
2023	186059.34	178534735.67

Insight: Some years had both higher-than-average pending amounts and higher average daily challan volumes. This helps identify years with greater challan activity and financial burden.

8.Find dates where the pending amount was higher than the disposed amount despite having more than 500 total challans.

```
SELECT date, totalChallan, pendingAmount, disposedAmount
FROM violations WHERE pendingAmount > disposedAmount
AND totalChallan > 500 ORDER BY pendingAmount DESC;
```



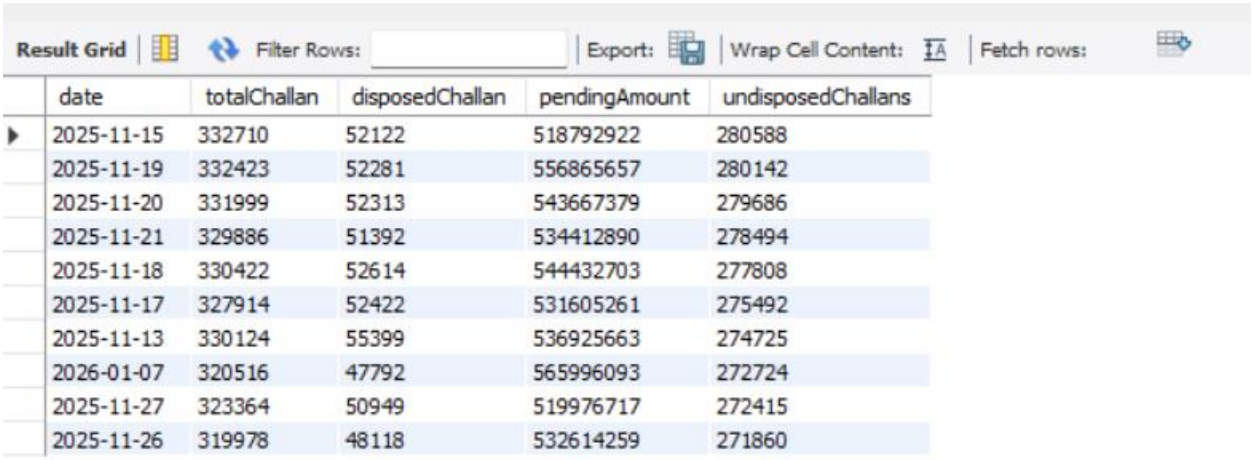
Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

date	totalChallan	pendingAmount	disposedAmount
2026-01-07	320516	565996093	93149973
2025-11-19	332423	556865657	99373534
2025-12-18	297905	555481448	85986419
2025-12-17	293594	544545088	92535332
2025-11-18	330422	544432703	95814517
2025-11-20	331999	543667379	89981341
2025-12-22	303202	543022969	81699743
2025-12-23	295607	537106063	86415551
2025-11-13	330124	536925663	99809765
2025-12-19	294971	534912879	85649663
2026-01-08	311700	534442793	95496534
2025-11-21	329886	534412890	84167178

Insight: Some dates had pending amounts higher than disposed amounts despite recording more than 500 total challans. This helps identify periods with high challan activity and greater unresolved financial burden.

9.Display the 10 dates with the largest gap between total and disposed challans among records with above-average pending amounts.

```
SELECT date, totalChallan, disposedChallan, pendingAmount,
(totalChallan - disposedChallan) AS undisposedChallans
FROM violations WHERE pendingAmount > ( SELECT
AVG(pendingAmount) FROM violations ) ORDER BY undisposedChallans
DESC LIMIT 10 ;
```



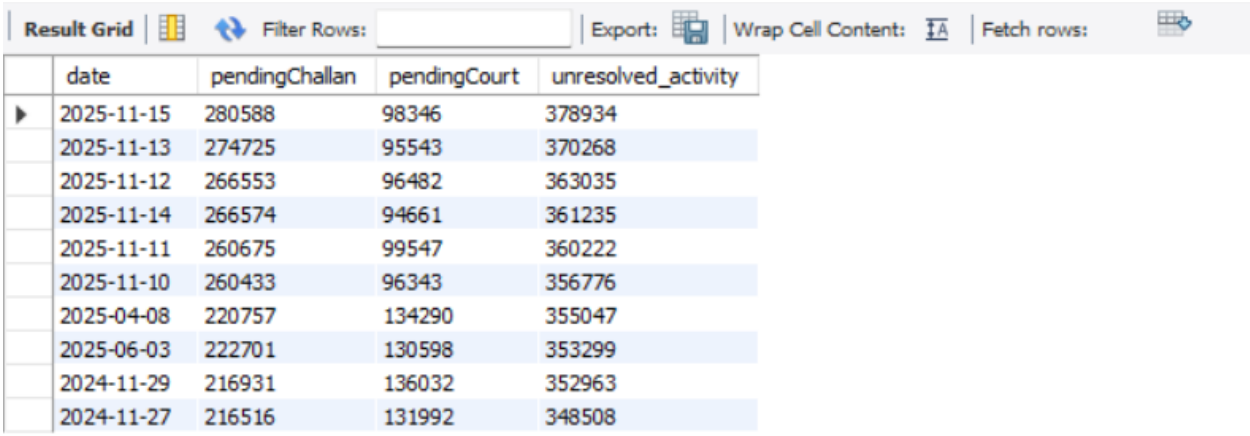
Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

date	totalChallan	disposedChallan	pendingAmount	undisposedChallans
2025-11-15	332710	52122	518792922	280588
2025-11-19	332423	52281	556865657	280142
2025-11-20	331999	52313	543667379	279686
2025-11-21	329886	51392	534412890	278494
2025-11-18	330422	52614	544432703	277808
2025-11-17	327914	52422	531605261	275492
2025-11-13	330124	55399	536925663	274725
2026-01-07	320516	47792	565996093	272724
2025-11-27	323364	50949	519976717	272415
2025-11-26	319978	48118	532614259	271860

Insight: Certain dates had both a large gap between total and disposed challans and above-average pending amounts. This helps identify periods with high unresolved challan activity and financial burden.

10.Display the 10 dates with the highest combined pending challan and pending court-case activity.

```
SELECT date, pendingChallan, pendingCourt,
(pendingChallan + pendingCourt) AS unresolved_activity
FROM violations ORDER BY unresolved_activity DESC LIMIT 10;
```

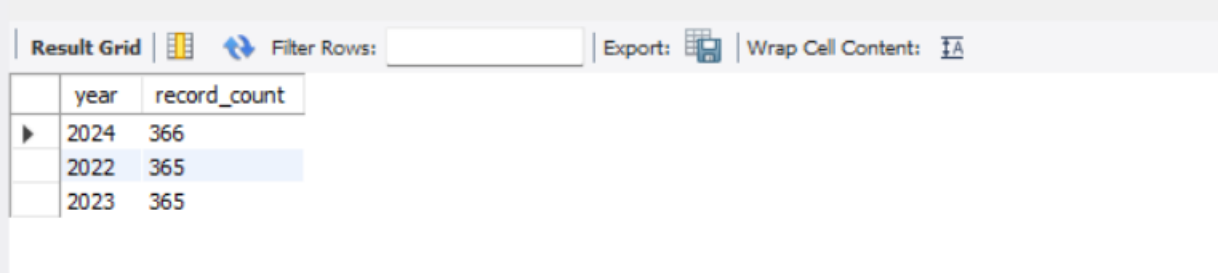


date	pendingChallan	pendingCourt	unresolved_activity
2025-11-15	280588	98346	378934
2025-11-13	274725	95543	370268
2025-11-12	266553	96482	363035
2025-11-14	266574	94661	361235
2025-11-11	260675	99547	360222
2025-11-10	260433	96343	356776
2025-04-08	220757	134290	355047
2025-06-03	222701	130598	353299
2024-11-29	216931	136032	352963
2024-11-27	216516	131992	348508

Insight: Some dates recorded particularly high combined pending challans and pending court cases. This helps identify periods with greater overall unresolved enforcement activity.

11.Compare the number of eChallan records across the selected years 2022, 2023, and 2024.

```
SELECT YEAR(date) AS year, COUNT(*) AS record_count FROM violations
WHERE YEAR(date) IN (2022, 2023, 2024) GROUP BY YEAR(date)
ORDER BY record_count DESC;
```



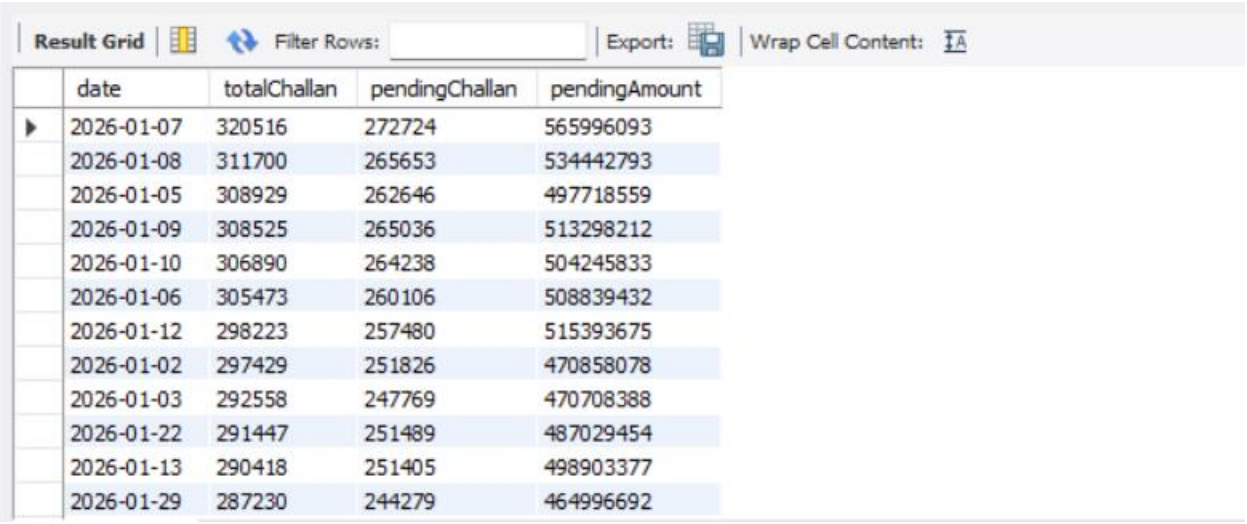
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

year	record_count
2024	366
2022	365
2023	365

Insight: The number of eChallan records differed across 2022, 2023, and 2024. This helps compare the recorded enforcement activity among the selected years.

12. Find January dates with more than 500 total challans and display them in descending order of challan volume.

SELECT date, totalChallan, pendingChallan, pendingAmount FROM violations WHERE DATE_FORMAT(date, '%Y-%m-%d') LIKE '%-01-%' AND totalChallan > 500 ORDER BY totalChallan DESC;



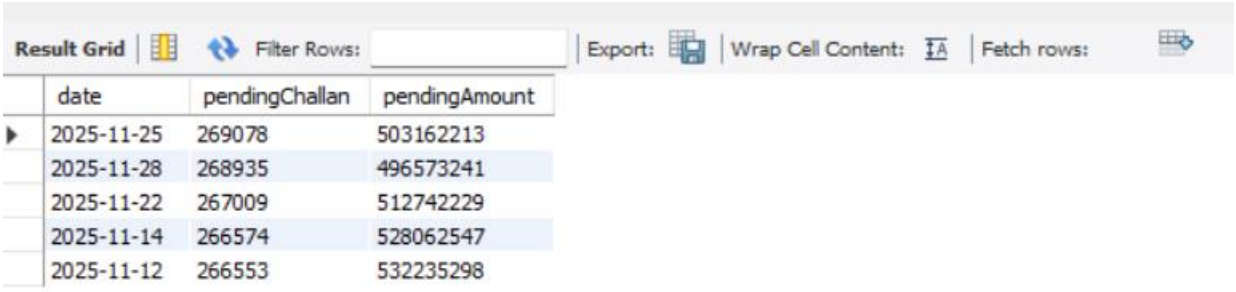
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

date	totalChallan	pendingChallan	pendingAmount
2026-01-07	320516	272724	565996093
2026-01-08	311700	265653	534442793
2026-01-05	308929	262646	497718559
2026-01-09	308525	265036	513298212
2026-01-10	306890	264238	504245833
2026-01-06	305473	260106	508839432
2026-01-12	298223	257480	515393675
2026-01-02	297429	251826	470858078
2026-01-03	292558	247769	470708388
2026-01-22	291447	251489	487029454
2026-01-13	290418	251405	498903377
2026-01-29	287230	244279	464996692

Insight: Several January dates recorded more than 500 total challans, with some dates showing higher challan volumes than others. This helps identify high-activity periods during January.

13. Display 5 dates ranked by pending challans after skipping the first 10 records, along with their pending amounts..

```
SELECT date, pendingChallan, pendingAmount FROM violations
ORDER BY pendingChallan DESC LIMIT 5 OFFSET 10;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

date	pendingChallan	pendingAmount
2025-11-25	269078	503162213
2025-11-28	268935	496573241
2025-11-22	267009	512742229
2025-11-14	266574	528062547
2025-11-12	266553	532235298

Insight: After excluding the 10 dates with the highest pending challans, the next five dates still showed significant pending activity. This helps examine dates ranking immediately below the highest pending-challan records.

14. Count the number of dates where pending challans exceeded disposed challans for each year.

```
SELECT YEAR(date) AS year,  COUNT(*) AS
dates_with_higher_pending_challans FROM violations
WHERE pendingChallan > disposedChallan
GROUP BY YEAR(date) ORDER BY dates_with_higher_pending_challans
DESC;
```

year	dates_with_higher_pending_challans
2016	366
2024	366
2023	365
2025	365
2015	364
2020	302
2021	245
2017	210
2022	148
2019	47
2026	44
2018	3

Insight: The number of dates where pending challans exceeded disposed challans varied across years. This helps identify years with more frequent unresolved challan situations.

15.Find the number of dates in each year where pending challans exceeded 100 and display the years in descending order of such dates.

SELECT YEAR(date) AS year, COUNT(*) AS high_pending_dates FROM violations WHERE pendingChallan > 100 GROUP BY YEAR(date) ORDER BY high_pending_dates DESC;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

year	high_pending_dates
2016	366
2020	366
2024	366
2017	365
2018	365
2021	365
2022	365
2023	365
2025	365
2019	364
2015	132
2026	44

Insight: The number of dates with more than 100 pending challans varied across years. This helps identify years that experienced more frequent periods of high pending-challan activity.

CONCLUSION

The eChallan Traffic Enforcement Analysis project successfully demonstrated the use of MySQL for analysing traffic violation, challan, financial, and court-case data to generate meaningful insights. Through the application of SQL concepts such as filtering, arithmetic operations, aggregation, GROUP BY, HAVING, ORDER BY, LIMIT, OFFSET, IN, LIKE, COUNT, and subqueries, the project transformed raw eChallan records into useful information.

The analysis provided insights into pending and disposed challans, undisposed challan activity, pending financial amounts, yearly challan volumes, and court-case activity. It also identified dates with higher unresolved challan activity and periods with greater pending financial and court-related workloads.

Additionally, the project highlighted yearly patterns in challan and court-case activity, helping understand changes in traffic enforcement and unresolved cases over time. The analysis of pending amounts and undisposed challans further supported the evaluation of financial and operational workload.

Overall, this project strengthened practical SQL skills and demonstrated how data analysis techniques can be applied to eChallan data to support data-driven understanding of traffic enforcement, pending cases, financial burden, and operational activity.